

Data-Driven Design Optimization — Generative-Based

Don't search the design — produce it

Jinkyoo Park · KAIST

Offline, part 2 of 2 — model the inverse, then sample

— HANDOFF

The handoff — the same problem, read backwards

Lecture 5 handed on a forward model and a search. Keep the problem; reverse the arrow.

Where we are — the sentence Lecture 5 handed over

STAGES **static** ● — ○

MODEL **data-driven** ○ — ●

AGENTS **single agent** ● — ○

MODEL-BASED (CERTAIN)		DATA-DRIVEN (UNCERTAIN)
Static, single	optimisation (Lec 1)	belief (Lec 2–3) · acting on belief (Lec 4) · design from a fixed dataset (Lec 5 ✓ · Lec 6)

What Lecture 5 handed on is **a forward model then a search, and the warning that the optimiser is an adversary**. It built $f_{\theta}(x) \approx f(x)$, ran an optimiser over it, and spent the whole lecture stopping that optimiser from exploiting the places where f_{θ} was wrong.

This lecture keeps every word of the problem and **reverses the arrow of the model**. Same cell of the cube — static, data-driven, single agent — and the same $\arg \max$. What changes is which direction we learn.

The problem, unchanged — and the route not taken

The source deck opens Lecture 6 with the identical slide it opened Lecture 5 with. Not an accident: it is one module in two parts.

$$\text{find } x^* = \arg \max_x f(x) \quad \text{with \textcolor{blue}{only} a fixed dataset } D = \{(x_1, f(x_1)), \dots, (x_N, f(x_N))\}$$

1 • SURROGATE-BASED — LECTURE 5

Approximate $f(x)$ from the collected data; choose the query that maximises the surrogate.

A **forward** model, and a **search**.

2 • GENERATIVE-BASED — THIS LECTURE

Learn an inverse function $f^{-1}(y)$ from the collected data; choose the query the inverse **generates from a desired output**.

An **inverse** model, and a **draw**.

And the deck is candid about why the second route exists. Conservatism — Lecture 5's cure — buys safety by **limiting the expressivity of the model**, and a model that has been taught to distrust itself often fails to find a better design at all. The generative route pays for safety differently.

The thesis — model what good designs look like, and draw one

Instead of asking "*which input scores highest?*", model "*what do high-scoring inputs look like?*" — and draw one.

Formally: jointly model the pair, then condition on the outcome you want.

$$p(x, y) = p(y \mid x) p(x) \quad \implies \quad x \sim p(x \mid y \geq y_{\max}) \quad (\text{generally } p(x \mid S))$$

Two objects, and they are the two halves of the lecture: a **generative model** of valid designs — the prior $p(x)$, learned so that samples land on the narrow valid manifold — and an **inversion** mechanism, which conditions that prior on a good outcome.

Three things sampling buys that searching cannot

the source deck's own list, and the reason the second route is not merely the first one backwards

-
- **No optimisation to solve.** There is no ascent to run, so there is no ascent that can run away. You draw a sample; that is the whole decision.
-
- **Diversity, for free.** A generative model is multi-modal by construction, so one query returns **a spread of distinct good designs**, not one point. An $\arg \max$ returns one answer even when the problem has several.
-
- **The constraints come for nothing.** The model was trained only on feasible designs, so **every sample is on the manifold by construction** — which is precisely the failure Lecture 5 could only mitigate.
-

Lecture 5 fights to stay on the valid manifold. Lecture 6 **never leaves it**, because it never moves through input space at all.

The roadmap — four questions

Q1 What is the inverse?

Q2 How do we model designs?

Q3 What is a VAE?

Q4 How do we steer?

- **Q1 — What does inverting the function mean?** Learn $p(x | y)$; train it by matching a divergence.
- **Q2 — How do we model valid designs at all?** **Generative models** — a prior $p(x)$ whose support is the manifold.
- **Q3 — One concrete model, worked through.** The **VAE**: encoder, decoder, and a latent space you can sample.
- **Q4 — How do we steer toward *good* designs?** **Conditioning** — CbAS, and Model Inversion Networks.

— ACT 1

Inverting the function

Q1. The forward map answers a question nobody asked. Turn it round.

The inverse map — from a desired outcome back to a design

Q1 What is the inverse?

Q2 How do we model designs?

Q3 What is a VAE?

Q4 How do we steer?

$$f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y} \quad \Longrightarrow \quad f_{\theta}^{-1} : \mathcal{Y} \rightarrow \mathcal{X}$$

The forward model answers *"given x , what is y ?"*. We never wanted that. We wanted *"given the y I need, what x delivers it?"* — and that is a map whose **input is a scalar**.

WHAT THE REVERSAL FIXES

The search was hard because $\mathcal{X}$ is enormous. The inverse map is not asked to search $\mathcal{X}$; it is asked to *emit* a point of it, and its own input is one number.

WHAT THE REVERSAL DOES NOT FIX

The **output** is still high-dimensional and still confined to a thin manifold. Reversing the arrow moves the difficulty; it does not remove it.

That surviving difficulty is exactly what Acts 2 and 3 are for: the inverse map's *output* head must be a generative model. The source deck says so on the same slide it announces the inversion — *"we can exploit various techniques from deep generative models to model high-dimensional data distributions."* (Kumar & Levine, MINs, NeurIPS 2020)

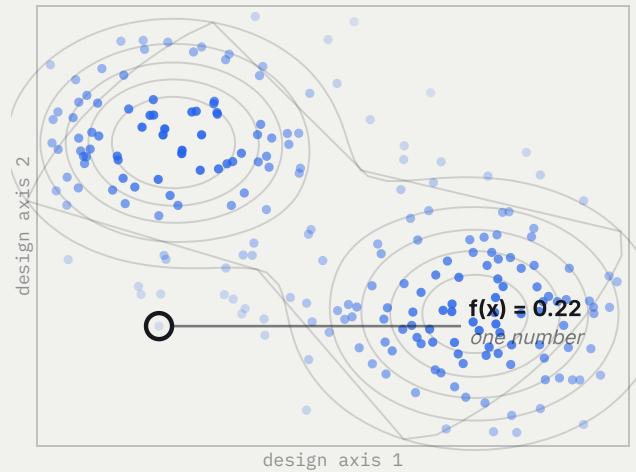
One y , many x — so the inverse cannot be a function

The forward map is single-valued: one design, one score. Run it backwards and it is **not a function at all** — a given performance is achieved by a whole set of designs, often in several disconnected clusters.

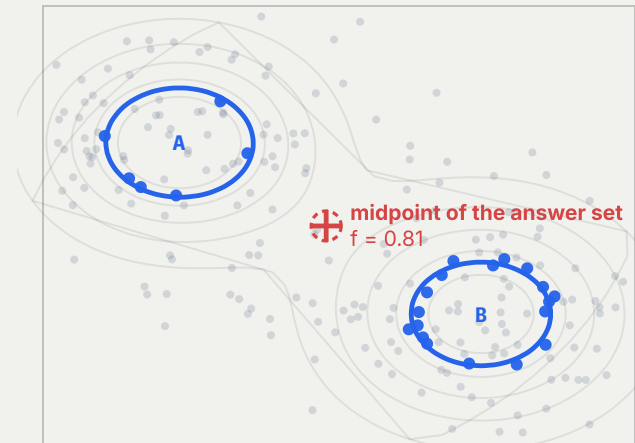
So the inverse map must be **stochastic**: $f_{\theta}^{-1} : \mathcal{Y} \times \mathcal{Z} \rightarrow \mathcal{X}$, with $z \sim p_0(z)$ supplying the choice among the many valid answers.

The answer set, and the point in the middle of it

220 DESIGNS, ONE LANDSCAPE, TWO DIRECTIONS



FORWARD $x \rightarrow y$ — ONE DESIGN, ONE SCORE



INVERSE $y \rightarrow \{x\}$ — ONE SCORE, MANY DESIGNS

Every point of that curve is worth exactly 2.00, and 24 designs in D sit on it (6 near A, 18 near B). Its midpoint is a design worth 0.81 — what a deterministic inverse map returns. Best in D: 2.72.

Read left to right and it is a function; read right to left and the answer is **a curve in two disconnected pieces**. The red mark is that set's midpoint: every point of the curve is worth 2.00, and their average is worth 0.81, stranded in the valley between the branches — which is exactly what a least-squares inverse map returns.

Training the inverse — pick a divergence, and a familiar objective appears

Train f_{θ}^{-1} to match the data's own conditional, by minimising a divergence at every outcome level:

$$L_p(D) = \mathbb{E}_{y \sim p(y)} \left[\text{Div} \left(p_D(x | y), p_{f_{\theta}^{-1}}(x | y) \right) \right]$$

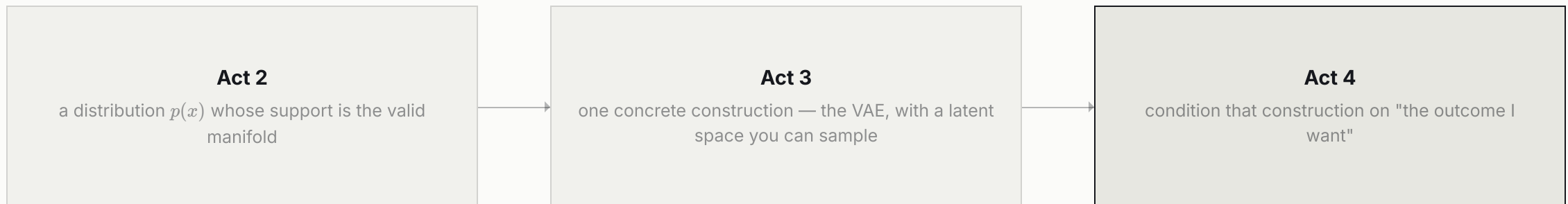
CHOICE OF DIV	WHAT THE OBJECTIVE BECOMES
KL divergence	maximum-likelihood training of the inverse map
JS divergence	a GAN-style adversarial objective

Nothing here is new machinery — it is the same "choose a divergence, get a loss" move that turns up whenever a distribution is fitted to another. What is new is the *object* being fitted: not a function, but **a conditional distribution over designs**.

Searching has been replaced by **conditioning**: set y high, and read off x .

What is still missing — a model that knows what a valid x looks like

The inversion is a change of question. It is not yet a method, because we have not said how a distribution over high-dimensional designs is represented at all.



Read that chain backwards and it is the definition of $p(x | y)$: a prior over designs, an oracle $p(y | x)$, and Bayes' rule. Act 4 will write exactly that line.

— ACT 2

Generative models

Q2. Before you can sample a *good* design, you must be able to sample a design at all.

The goal — a distribution whose support is the valid manifold

Q1 What is the inverse?

Q2 How do we model designs?

Q3 What is a VAE?

Q4 How do we steer?

THE GOAL

Train $p_{\theta}(x)$ to match the data distribution $p_{\text{data}}(x)$, given only samples $\{x_i\}_{i=1}^N \sim p_{\text{data}}$.

A generative model maps simple noise $z \sim \mathcal{N}(0, I)$ through a network to a realistic sample x . The source figure draws the data as a ragged starfish-shaped blob in design space and the model's output as a second blob being pulled onto it: what is being learned is **the shape of the manifold**, not merely the location of its centre.

Which is the whole point for us. Lecture 5's second failure — "only a thin sliver of input space is valid, and ascent leaves it almost immediately" — is answered here by *what the model is*, before any optimisation is discussed.

Fitting it — minimising the divergence **is** maximum likelihood

Learning a generative model means picking the member of the model family closest to the data distribution. Write that down and it collapses into something you already know:

$$\arg \min_{p_\theta} \mathbb{D}_{\text{KL}}(p_{\text{data}} \| p_\theta) = \max_{p_\theta} \frac{1}{|D|} \sum_{x_i \in D} \log p_\theta(x_i)$$

The KL divergence $\mathbb{D}_{\text{KL}}(P \| Q) = \mathbb{E}_{x \sim P}[\log P(x)/Q(x)]$ measures the inefficiency of using Q in place of P . Its first term does not involve θ , so minimising it is maximising the average log-likelihood of the data — **Act 1's "KL $\Rightarrow$ maximum likelihood" row, arriving one act early and for the same reason.**

So the whole enterprise is Lecture 2's likelihood principle applied to a model with no closed-form likelihood. Everything difficult about generative modelling is the phrase $\log p_\theta(x_i)$ for a high-dimensional x_i .

The latent-variable move

Learning $p_\theta(x)$ directly for a high-dimensional x is hard. So do not: assert that the data is generated by a few simple hidden factors, and model *those*.

$$p(x) = \int_z p(x | z) p(z) dz$$

THE STORY

The data is governed by a simple latent distribution $p(z)$ — the source deck draws six axes labelled *smile, skin tone, gender, beard, glasses, hair colour*, each a plain Gaussian. The observed x is generated from z by a conditional $p(x | z)$: a decoder.

THE TRANSLATION TO DESIGN

The latent axes are the **degrees of freedom a valid design actually has**. A protein has far fewer meaningful degrees of freedom than it has atoms; a truss has far fewer than it has nodes. The generative model's job is to find that coordinate system.

Note what this buys before any optimisation happens. A design expressed in latent coordinates is valid **for every value of z** , because the decoder was only ever trained to emit valid designs. The manifold has become the whole space.

Four families, one job

FAMILY	MECHANISM	LATENT	TRADE-OFF
Variational autoencoder	encode to a latent, decode; maximise a variational bound	explicit, $\sim \mathcal{N}(0, I)$	stable; samples can be blurry
GAN	generator against discriminator	implicit	sharp; unstable, mode collapse
Normalising flow	invertible transform of a simple density	invertible, same dimension	exact density; architectural constraints
Diffusion	add Gaussian noise, then learn to reverse it	fixed noise schedule	high quality; slow sampling

Any of the four can serve as the prior $p(x)$ in a generative design pipeline, and the papers in Act 4 use different ones. We take the **VAE** as the worked example because it makes the latent variable — and therefore the inversion — most explicit, and because the other three are best understood as departures from it.

— ACT 3

The VAE

Q3. One model, worked through: encoder, decoder, and the term that makes the code samplable.

The evidence is intractable — so bound it

Q1 What is the inverse?

Q2 How do we model designs?

Q3 What is a VAE?

Q4 How do we steer?

We want to maximise $\log p_\theta(x)$, but it integrates over z . Introduce a variational posterior $q_\phi(z | x)$ and push the log inside:

$$\log p(x) = \log \int_z q_\phi(z | x) \frac{p(x, z)}{q_\phi(z | x)} dz \geq \mathbb{E}_{q_\phi} \left[\log \frac{p(x, z)}{q_\phi(z | x)} \right] = \underbrace{\mathbb{E}_{q_\phi} [\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z))}_{\text{ELBO}}$$

The gap is exactly one KL divergence, and it is non-negative:

$$\text{KL}(q_\phi(z | x) \| p(z | x)) = -\text{ELBO} + \log p(x) \geq 0$$

Maximising the evidence = maximising the ELBO = **driving q_ϕ onto the true posterior**. Three readings of one inequality.

Lecture 2 wrote a variational posterior over one parameter and solved it in closed form. This is that idea with a neural network in place of the closed form — and it is why the deck's own section heading for the VAE is "*Variational Inference*".

The architecture — encode, decode, generate

GENERATIVE PROCESS — HOW DESIGNS ARE PRODUCED

Draw $z \sim \mathcal{N}(0, I)$ from the prior, decode $x \sim p_\theta(x \mid z)$.

That is the whole sampler. It never touches the data, and it never runs an optimiser.

INFERENCE PROCESS — HOW THE CODE IS FOUND

The encoder $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$ approximates the intractable posterior $p(z \mid x)$.

It exists only to make training possible.

$$\max_{\phi, \theta} \frac{1}{N} \sum_{i=1}^N \log p_\theta(x_i \mid \underbrace{\mu_\phi(x_i) + \epsilon \sigma_\phi(x_i)}_{\text{reparameterised } z}) - \text{KL}[\mathcal{N}(\mu_\phi(x_i), \sigma_\phi(x_i)) \parallel \mathcal{N}(0, I)], \quad \epsilon \sim \mathcal{N}(0, I)$$

The reparameterisation moves the randomness off the parameters and onto ϵ , so a gradient can flow through the sample. Without it there is nothing to differentiate. (*Backup 1 derives both the bound and the trick.*)

Reading the ELBO — reconstruct, and stay regular

$$\mathcal{L} = \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)]}_{\text{reconstruction}} - \beta \underbrace{\text{KL}(q_\phi(z | x) \| p(z))}_{\text{stay near the prior}}$$

B TOO SMALL — THE CODE CHEATS

The encoder shrinks σ_ϕ to nothing, so each design gets a private, isolated point. Reconstruction is perfect and the code means nothing between the points: draw z from the prior and the decoder has never been there.

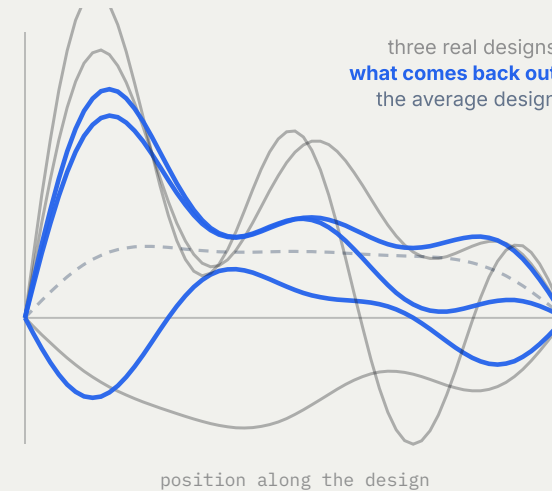
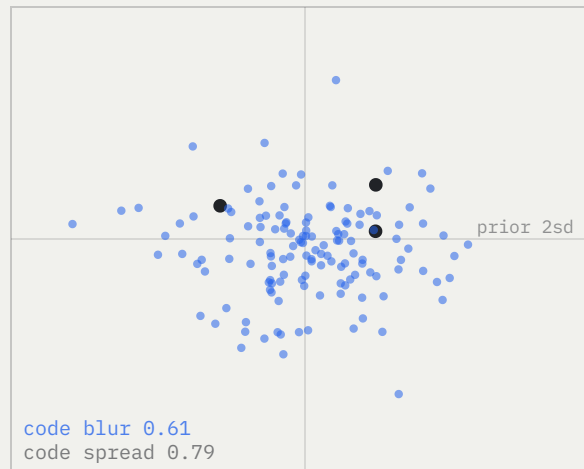
B TOO LARGE — THE CODE COLLAPSES

The KL term wins outright: $\mu_\phi(x) \rightarrow 0$ for every design and $\sigma_\phi \rightarrow 1$. The posterior *is* the prior, the code carries no information, and every z decodes to the average design.

The KL term is what makes the latent space **samplable**: a smooth, prior-shaped code from which new valid designs can actually be drawn.

Turning the dial

AN EXACTLY SOLVABLE VAE — THE KL WEIGHT, TURNED



THE LATENT CODE — DOTS ARE DESIGNS, RINGS ARE THE PRIOR

THREE DESIGNS, AND WHAT THE DECODER RETURNS

$$\text{code spread}^2 \mathbf{0.622} + \text{code blur}^2 \mathbf{0.378} = \mathbf{1.0000} \cdot \text{KL}(\text{aggregate} \parallel \text{prior}) \mathbf{0.0000} \cdot \text{reconstruction} \mathbf{0.643}$$

the aggregate code **is** the prior, exactly — every z the decoder will ever be handed is a z it has seen

An exactly solvable VAE, so every number is computed rather than fitted. Each dot is a design's code, each halo its blur, and the ring is the prior. **Spread and blur are two variances that sum to exactly 1.000 at $\beta = 1$ and at no other value** — 1.565 at $\beta = 0.05$, where the codes spill outside the prior, and 0.930 at $\beta = 20$, where they have collapsed to a dot.

Condition the encoder and the decoder, and the inverse map is built

the conditional VAE — the slide the compressed notes drop, and the hinge of the whole lecture

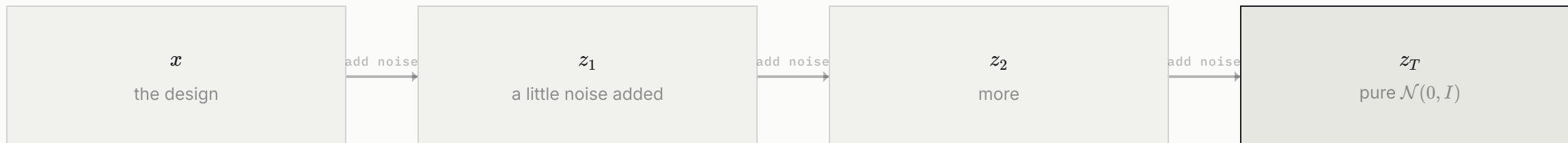
$$\max_{\theta, \phi} \mathbb{E}_{q_{\phi}(z|x,y)} [\log p_{\theta}(x | z, y)] - \text{KL}(q_{\phi}(z | x, y) \| p_{\theta}(z | y))$$

That is the entire modification: **feed the condition y into both networks**. The source deck's figure trains a CVAE on street-number photographs and shows a grid whose *row* is the digit you asked for and whose *column* is a style — the condition fixed, the remaining freedom sampled.

And then the deck says the quiet part out loud: **y can be the desired property, and x the corresponding design.**

So the inverse map of Act 1 is not a new architecture at all. It is a conditional generative model, with the condition set to the outcome we want and the noise z supplying the diversity that a one-to-many map requires. Act 4 is about which y to ask for — and that turns out to be the hard part.

Diffusion — a VAE whose encoder was never trained



Stack the VAE T times and you have a hierarchical latent model, $\mathcal{L}_{\theta, \phi}(x) = \mathbb{E}_{q_{\phi}(z_{1:T}|x)} [\log p_{\theta}(x, z_{1:T}) / q_{\phi}(z_{1:T} | x)]$. A diffusion model is that stack under three restrictions:

- the **latent dimension equals the data dimension** — no bottleneck;
- the encoder is a **fixed linear-Gaussian noising process**, $q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$ — **no parameters to train**, so ϕ vanishes from the objective;
- the top-level prior is set to $p(x_T) = \mathcal{N}(0, I)$.

That Gaussian kernel is not asserted; it is *derived*. The source deck asks what $q(x_t | x_{t-1})$ must satisfy — a closed form for $q(x_t | x_0)$, a closed form for $q(x_{t-1} | x_t, x_0)$, and a limit that forgets x_0 — and observes that the obvious distribution meeting all three is the Gaussian. *"This is the formulation introduced in DDPM."* (Ho, Jain & Abbeel, *NeurIPS 2020*) Backups 2 and 3 carry the derivation.

The score view, and a failure you have met before

why the same model can be trained by a bound or by a gradient field

An energy model $p_\theta(x) = e^{-\varepsilon_\theta(x)} / Z_\theta$ is untrainable by likelihood because Z_θ is intractable. But differentiate the *log density in x* and the constant disappears:

$$s_\theta(x) \approx \nabla_x \log p(x) = -\nabla_x \varepsilon_\theta(x) - \cancel{\nabla_x \log Z_\theta}$$

Fit s_θ by the Fisher divergence and sample by Langevin dynamics, $\tilde{x}_t = \tilde{x}_{t-1} + \frac{\epsilon}{2} \nabla_x \log p(\tilde{x}_{t-1}) + \sqrt{\epsilon} z_t$. Diffusion's own loss can be rewritten in exactly this form, which is why the two literatures are one.

BUT LOOK AT THE OBJECTIVE'S WEIGHT

and then look at Lecture 5 again

$$\mathbb{E}_{p(x)} [\|\nabla_x \log p(x) - s_\theta(x)\|^2] = \int p(x) \|\nabla_x \log p(x) - s_\theta(x)\|^2 dx$$

The error is weighted by $p(x)$, so it is **largely ignored wherever the data is thin**. The source figure shows it: two accurate patches over the two data clusters, and a large region between them where the estimated gradient field is simply wrong, and the training loss never noticed.

This is Lecture 5's thesis, in a different half of the subject: **a learned object is unconstrained where there is no evidence**. The cure is even the mirror image of conservatism — where Lecture 5 pushed the model *down* off-distribution, score-based models perturb the data with noise at several scales to push the data outwards until the empty region is populated, then anneal the noise away.

— ACT 4

Conditioning toward good designs

Q4. A prior gives valid designs. We want valid *and* high-scoring ones.

The query — set the bar high, and sample below it

Q1 What is the inverse?

Q2 How do we model designs?

Q3 What is a VAE?

Q4 How do we **steer**?

THE PIECES

- a **forward oracle** $p(y | x)$ — possibly a black box, possibly a network fitted to D : *"how good is this design?"*;
- a **generative prior** $p(x)$ over valid designs, from Acts 2–3;
- together they define the joint $p(x, y) = p(y | x)p(x)$, and hence the conditional.

THE QUERY

$$x \sim p(x | y \geq y_{\max})$$

Set the bar above anything in the data, sample below it, and out come candidate designs — on-manifold by construction.

$$\nabla_x \log p(x | y) = \underbrace{\nabla_x \log p(x)}_{\text{the prior over designs}} + \underbrace{\nabla_x \log p(y | x)}_{\text{the oracle}}$$

The whole lecture on one line: [an inverse problem is a Bayesian inference problem](#), and its score is the prior's score plus the oracle's.

Which is also how conditional image generation works: classifier guidance adds $s \Sigma \nabla_{x_t} \log p_\phi(y | x_t)$ to every denoising step, and the scale s is the dial. In the source deck's example, guidance scale 1.0 gives FID 33.0 and unconvincing samples; scale 10.0 gives FID 12.0 and class-consistent ones. [How hard to push the condition is a hyperparameter, and it matters.](#) (Dhariwal & Nichol, 2021)

CbAS — fit a generative model to its own best samples

Conditioning by Adaptive Sampling · Brookes, Park & Listgarten, ICML 2019

Write the target down with Bayes' rule, taking the *generative prior* as the prior and the *oracle* as the likelihood:

$$p(x \mid S, \theta^{(0)}) = \frac{P(S \mid x) p(x \mid \theta^{(0)})}{P(S \mid \theta^{(0)})}$$

We cannot sample that directly, so fit a second generative model $q(x \mid \phi)$ to it — and the fit has a closed form as an expectation *under the prior*:

$$\phi^* = \arg \min_{\phi} \text{KL}(p(x \mid S, \theta^{(0)}) \parallel q(x \mid \phi)) = \arg \max_{\phi} \mathbb{E}_{p(x \mid \theta^{(0)})} [P(S \mid x) \log q(x \mid \phi)]$$

Read the right-hand side as **weighted maximum likelihood**: draw designs from the model you have, weight each by how likely it is to satisfy the requirement, and refit. Which is a familiar shape — it has the flavour of variational inference, and q need not even be in the same family as the prior.

Why you cannot simply sample and reweight

The estimator above is unbiased and useless. In a design problem, satisfying S is **exceedingly rare**, so $P(S \mid x)$ is vanishingly small for almost every x drawn from the prior; the Monte-Carlo average is then dominated by a handful of samples and needs an arbitrarily large number of draws to be accurate.

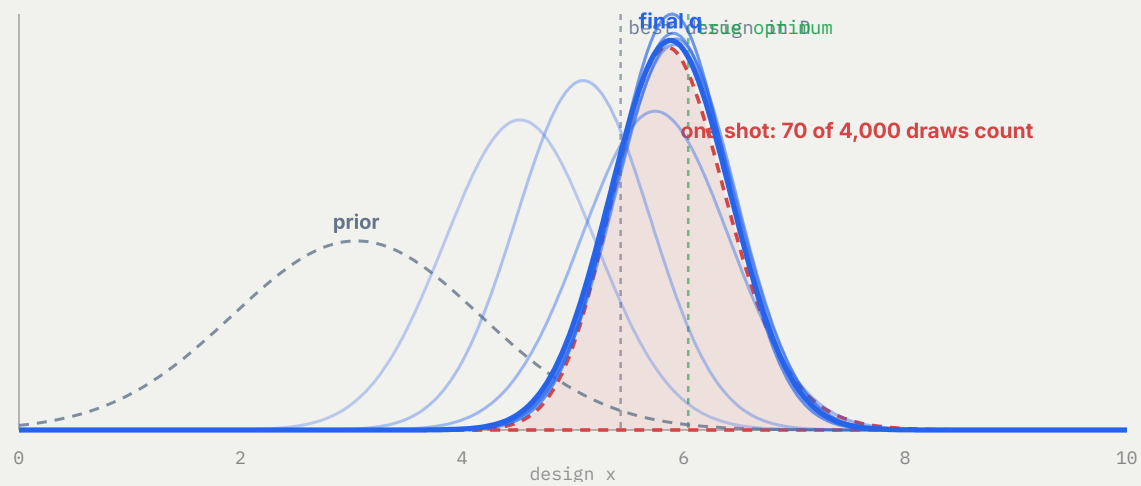
THE TWO CONDITIONS THE LADDER MUST SATISFY

- **(A)** $\mathbb{E}_{r^{(t)}}[P(S^{(t)} \mid x)]$ is **non-vanishing** — the current proposal can actually produce samples that meet the current bar;
- **(B)** $S^{(t)} \supset S^{(t+1)} \supset S$ for every t — the bar only ever rises, and reaches the real target in the limit.

The proposal is the previous iterate, $r^{(t)} = q(x \mid \phi^{(t-1)})$, so the importance weight is $p(x \mid \theta^{(0)})/q(x \mid \phi^{(t)})$ and the estimator stays low-variance throughout. **A hard conditional query, replaced by a sequence of easy ones.**

One shot, then the ladder

A HARD CONDITIONAL QUERY, REPLACED BY A SEQUENCE OF EASY ONES



THE RELAXED CONDITIONS

t	bar	q	useful
0	1.891	$N(3.04, 1.12^2)$	27%
1	2.239	$N(4.52, 0.68^2)$	47%
2	2.506	$N(5.09, 0.60^2)$	27%
3	2.650	$N(5.74, 0.66^2)$	88%
4	2.650	$N(5.91, 0.55^2)$	98%
5	2.650	$N(5.93, 0.54^2)$	99%
6	2.650	$N(5.90, 0.53^2)$	99%
7	2.650	$N(5.89, 0.51^2)$	98%

Straight to the target: **70** of 4,000 draws carry weight (1.76%).
The ladder's worst round kept **27%** — **16×** the one-shot rate.
Final **q = N(5.88, 0.54²)**; the design at its mean is worth **2.647** (best in D 2.523, true optimum 2.657)

Ask for the target in one shot — sample the prior, weight by $P(S | x)$ — and at $y_{\text{target}} = 2.65$ only **70 of 4,000 draws** carry any weight; at 3.00, eight do. Now run the ladder, relaxing the bar to the 85th percentile of the model's own predictions and tightening. **The worst round still keeps 27%** and the design it returns is worth 2.647, against the dataset's best of 2.523.

MINs — train the inverse map, and choose the y you ask for

Model Inversion Networks · Kumar & Levine, NeurIPS 2020

TRAINING — WHICH Y DO YOU LEARN ON?

Uniform sampling is straightforward, but the goal is optimisation and **accuracy at high y matters far more**. Greedy sampling on the single best y has crippling variance. So reweight:

$$\hat{\mathcal{L}}_p(D) = \frac{1}{|D|} \sum_i w_i \cdot \hat{D}(x_i, f_\theta^{-1}(y_i))$$

$$w_i = \frac{p(y)}{p_D(y)}, \quad p(y) \propto \frac{N_y}{N_y + K} e^{y - y^*}$$

Upweight high scores — **provided the number of samples at that score is not too low**. N_y is that count; y^* the best score in D .

QUERYING — HOW FAR PAST THE DATA DO YOU ASK?

Taking the best y in D caps the answer at what we already own. Instead, ask for the largest y the *forward* model will still agree with:

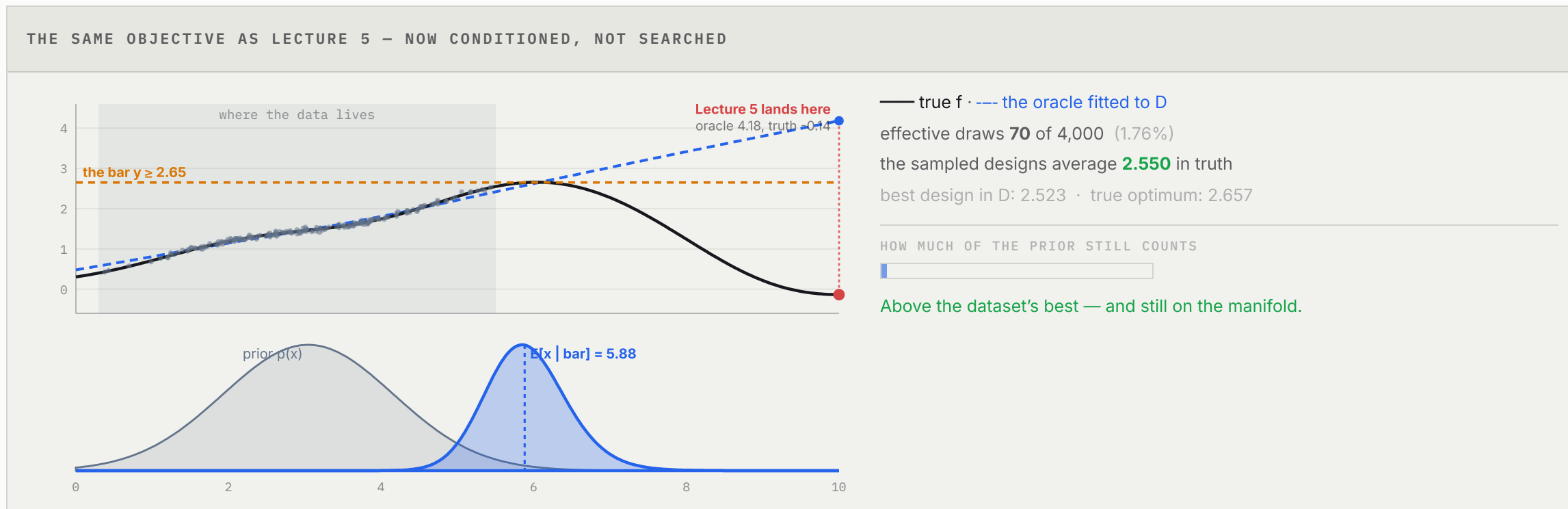
$$\tilde{y}^*, \tilde{z}^* = \arg \max_{y, z} f_\theta(f_\theta^{-1}(y, z))$$

$$\text{s.t.} \quad \|y - f_\theta(f_\theta^{-1}(y, z))\|^2 \leq \epsilon_1, \quad p(z) \geq \epsilon_2$$

The first constraint demands **self-consistency**; the second keeps z in the region the decoder understands. Together: *extrapolate as far as possible while staying on the data manifold*.

Note the shape. A budget ϵ_1 on how far the query may drift from what the forward model will confirm, and a floor ϵ_2 on prior density, is **the trust region of Lecture 1** in a new costume — the same move Lecture 5 made when it turned COMs' penalty α into a constraint τ .

How far past the data can you ask?



The same two hundred designs, and the same oracle, now driven by the bar γ alone. Raise it and the design distribution shifts toward the good region: at $\gamma = 2.00$ the samples average a true value of 2.13; at $\gamma = 2.65$ they average 2.550, above the best design in the dataset at 2.523. Raise it further and they **thin out and then decay**: 2.537 at $\gamma = 2.80$, 2.427 at 3.00, 2.163 at 3.80 — while the effective sample count falls from 3,750 to 1.5. The grey marker is Lecture 5's answer on this identical problem: gradient ascent on the same oracle runs to the boundary and returns a design the oracle rates 4.18 and the world rates **-0.14**.

The prior is the leash. The oracle is just as wrong out there as it was in Lecture 5 — but **the sampler cannot go where the prior has no mass**, so the failure is a loss of precision rather than a hallucination.

What it does on real design problems

MINs, three tasks · note the third column

TASK	DIMENSION	DATASET AVG	DATASET BEST	FORWARD MAP	MIN
MNIST, thickest recognisable "3"	1,024	149.0	265.0	<i>Invalid</i>	276.3
MNIST, variant (b)	1,024	149.0	163.0	<i>Invalid</i>	234.3
Faces, youngest (≥ 15)	12,288	38.7	−15.0	<i>Invalid</i>	−12.2
Faces, youngest (≥ 25)	12,288	41.5	−25.0	<i>Invalid</i>	−23.9
HopperController reward	3,843	442.9	1915.5	93.1	1960.1
Pendulum reward	1,537	14.7	344.5	3.4	1000.0

The *forward map* column is Lecture 5's naive route, and on four of six tasks its answer is not a wrong design but **not a design at all** — a 32×32 array of pixels that is not an image of a digit, a 64×64 array that is not a face. On Hopper it is a controller scoring 93 where the dataset already contained one scoring 1915. Meanwhile MIN beats the dataset's best on five of six. (*Kumar & Levine, 2020*)

The 2023 descendants

what happens when the generative model gets better, and when it does not

DDOM — DIFFUSION AS THE INVERSE MAP (ICML 2023)

The inverse map is one-to-many; a unimodal model averages the modes. So make it a **conditional diffusion model**, trained on the MINs weights and sampled with classifier-free guidance:

$$\epsilon_{\theta}(x, t, y) = (1 + \gamma) \epsilon_{\text{cond}}(x, t, y) - \gamma \epsilon_{\text{uncond}}(x, t)$$

On a three-mode test problem the reverse trajectory splits and recovers **all three**. Best mean rank on Design-Bench: 2.8, against COMs' 3.7 and gradient ascent's 3.5.

BOOTGEN — AND THE FAILURE RETURNS (KIM, BERTO, AHN & PARK, NEURIPS 2023)

Generative modelling *"sometimes gives us bad results — lack of generalisability on high-scoring regions."* The stated reason: **exploiting a single trained model may be dangerous, and fall into out-of-distribution.**

The answer is Lecture 5's, twice over: **rank-based** reweighting, so the weights do not depend on the objective's scale; and **bootstrapping** — several generators, each augmenting its training set with its own top- K samples as scored by a proxy. Average 0.895 against the dataset's 0.365 and the best baseline's 0.792, with the **highest diversity and novelty** of any method tested.

Read the right-hand column again. The generative half, pushed hard enough, meets **the optimiser-as-adversary** all over again — and answers it with an ensemble and a conservative re-ranker. The two routes of Part III do not stay separate for long.

— CLOSING

Closing

An inverse model, sampled. And a duality that is about to return one level up.

Why generation sidesteps the surrogate trap

Lecture 5's failure was an ascent that climbed off the valid manifold into hallucinated peaks. The generative approach removes that failure **structurally**:

- it never optimises *over* input space, so there is no ascent to run off-manifold;
- every output is a *sample* from a model of *valid* designs — on-manifold by definition;
- conditioning on $y \geq y_{\max}$ steers toward good designs **within** that valid set.

THE TRADE

neither route dominates

Surrogate search risks invalid designs but optimises sharply, and can in principle reach anywhere.

Generative sampling guarantees valid designs but is **bounded by what the data's good region contains**: ask for more than the prior can supply and the samples thin out, then decay.

Which is why a practitioner uses both, and why the field's best current methods are hybrids: a generative prior to stay valid, a conservative surrogate to rank. Lecture 5's LCOMs did it from the other side — a crystal-structure VAE supplying the coordinates, and a conservative surrogate optimised *inside* that latent space.

Where we are — the design-optimisation duality, complete

	SURROGATE (LEC 5 ✓)	GENERATIVE (LEC 6 ✓)
direction	forward $f_{\theta}(x)$	inverse $p(x y)$
the decision	search for the maximum	sample a design
the valid manifold	guarded, by conservatism	enforced, by construction
the risk	off-manifold hallucination	limited to the data's good region
what fails first	the model's honesty	the data's coverage

What this lecture hands on is **an inverse model then sampling**: model $p(x | y)$, condition it on the outcome you want, and draw. Search against produce — the same problem, attacked from opposite ends.

The rhyme that is about to return

Hold the shape of these two lectures. It is not a detail of offline optimisation; it is the deepest split in reinforcement learning, seen early and in miniature.

SEARCH A VALUE — LECTURES 5, 8

Learn a **forward** model of quality and then optimise over the decision:

$$a^* = \arg \max_a Q(s, a)$$

Value-based RL. It inherits Lecture 5's hazard exactly: the policy exploits Q where Q is wrong.

PRODUCE AN ACTION — LECTURES 6, 10

Learn an **inverse** map that emits a good decision directly, and sample it:

$$a \sim \pi_\theta(a \mid s)$$

Policy-based RL. Lecture 10's thesis is this lecture's, one axis over: **if you cannot search for the best action, learn to output it.**

And Part III closes here, and with it the static half of the course. Every problem so far — a known f , a belief over f , a queried f , a fixed dataset of f — has been **a single decision**. Lecture 7 lets the decision unfold over time, the model returns for one lecture, and the value function is born.

Don't search the design — produce it.

To optimise a function you only have as data, you can approximate it and **search** (Lecture 5) — or model its inverse and **sample** (Lecture 6). Forward-and-search, or inverse-and-produce: the two ways to turn data into a decision.

Questions?

Two things to carry out of here. **Conditioning replaces searching** — and a sample from a model of valid designs cannot be a hallucination, only a disappointment. And **search against produce** — you will meet it again in Lecture 10, wearing $Q(s, a)$ and $\pi_\theta(a \mid s)$.

— APPENDIX

Backup slides

Complete derivations, kept out of the narrative.

Backup 1 — the ELBO, derived, and the reparameterisation trick

We want $\log p_\theta(x)$, which integrates over z . Introduce $q_\phi(z | x)$ and apply the KL identity:

$$\log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right]}_{\text{ELBO } \mathcal{L}_{\theta, \phi}(x)} + \underbrace{\text{KL}(q_\phi(z | x) \| p_\theta(z | x))}_{\geq 0}$$

Since the KL is non-negative the ELBO is a lower bound, and maximising it both fits the data and drives q_ϕ toward the true posterior. Expanding $p_\theta(x, z) = p_\theta(x | z)p(z)$ gives the working form

$$\mathcal{L}_{\theta, \phi}(x) = \mathbb{E}_{q_\phi(z | x)}[\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)) \quad \blacksquare$$

Reparameterisation. To backpropagate through $z \sim q_\phi$, write $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$: the randomness moves off the parameters and the gradient flows.

Why the KL term is exactly the samplability term. With a linear-Gaussian decoder the β -optimal encoder is $\mu = (W^\top W + \beta\sigma^2 I)^{-1} W^\top (x - b)$ and $\Sigma = \beta\sigma^2 (W^\top W + \beta\sigma^2 I)^{-1}$. Along an eigendirection of $W^\top W$ with eigenvalue w the aggregate posterior variance is $\text{Var}(\mu) + \Sigma = (w^2 + \sigma^2 w + \beta\sigma^2 w + \beta^2\sigma^4)/(w + \beta\sigma^2)^2$, whose difference from 1 is $\sigma^2 w(1 - \beta)/(w + \beta\sigma^2)^2$ — zero **if and only if** $\beta = 1$. The Act 3 widget is that formula, plotted.

Backup 2 — the four families, and diffusion as a hierarchical VAE

FAMILY	TRAINING	LATENT	TRADE-OFF
VAE	maximise the ELBO	explicit, $\sim \mathcal{N}(0, I)$	stable; samples can be blurry
GAN	adversarial, generator vs. discriminator	implicit	sharp; unstable, mode collapse
Normalising flow	exact maximum likelihood	invertible, same dimension	exact density; architectural constraints
Diffusion	denoising score matching	fixed noise schedule	high quality; slow sampling

Backup 2b — diffusion as a hierarchical VAE

Forward and reverse. $q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$ and $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)$, and the same ELBO argument gives

$$\mathcal{L}(\theta) = \mathbb{E}_q \left[\underbrace{\text{KL}(q(x_T | x_0) || p_\theta(x_T))}_{L_T} + \sum_{t=2}^T \underbrace{\text{KL}(q(x_{t-1} | x_t, x_0) || p_\theta(x_{t-1} | x_t))}_{L_{t-1}} - \underbrace{\log p_\theta(x_0 | x_1)}_{-L_0} \right]$$

Why Gaussian. The kernel is chosen so that three things hold: $q(x_t | x_0)$ has a closed form; $q(x_{t-1} | x_t, x_0)$ has a closed form; and $\lim_{T \rightarrow \infty} q(x_T | x_0)$ forgets x_0 . With $q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$, $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i \leq t} \alpha_i$, all three follow:

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad q(x_{t-1} | x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t, \quad \bar{\alpha}_t \rightarrow 0 \Rightarrow q_\infty = \mathcal{N}(0, I)$$

Backup 3 — the DDPM objective collapses to one squared error

Both $q(x_{t-1} \mid x_t, x_0)$ and $p_\theta(x_{t-1} \mid x_t)$ are Gaussian, so L_{t-1} is an analytic KL between two Gaussians:

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t) \right\|^2 \right] + C$$

Substituting $x_t(x_0, \epsilon) = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ shows what μ_θ must predict, and suggests predicting the *noise* instead:

$$\mu_\theta \stackrel{\text{def}}{=} \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \quad \implies \quad L_{t-1} = \mathbb{E}_{x_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left\| \epsilon - \epsilon_\theta(x_t, t) \right\|^2 \right]$$

Dropping the weight — which the authors found works better in practice — leaves the whole evidence bound as a single regression:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t) \right\|^2 \right], \quad t \sim U(1, T)$$

The other algorithmic choices: schedule β_t as constants rather than learning them; fix $\Sigma_\theta(x_t, t) = \sigma_t^2 I$ with $\sigma_t^2 = \beta_t$ or $\tilde{\beta}_t$; learn μ_θ only, and learn it as an error term. Sampling is $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$. (Ho, Jain & Abbeel, 2020)

Backup 4 — score matching, Langevin dynamics, and the SDE that unifies them

The estimator. Fit $s_\theta(x) \approx \nabla_x \log p(x)$ by the Fisher divergence. The data score is unknown, but there are two equivalent computable forms:

$\mathbb{E}_p[\text{tr}(\nabla_x s_\theta) + \frac{1}{2}\|s_\theta\|^2]$ (Hyvärinen, 2005) and the denoising form $\frac{1}{2}\mathbb{E}_{q_\sigma(\tilde{x}|x)p(x)}[\|\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} | x) - s_\theta(\tilde{x})\|^2]$ (Vincent, 2011).

The blind spot, and its cure. The Fisher divergence weights the error by $p(x)$, so it is blind in low-density regions. Fix it by training on data perturbed at L noise scales, $p_{\sigma_i}(x) = \int p(y)\mathcal{N}(x; y, \sigma_i^2 I)dy$, with a noise-conditional model $s_\theta(x, i)$ and loss $\sum_i \sigma_i^2 \mathbb{E}_{p_{\sigma_i}} \|\nabla \log p_{\sigma_i} - s_\theta(x, i)\|^2$; then sample by **annealed Langevin dynamics**, running the chain for $i = L, L - 1, \dots, 1$.

The continuous limit. Let the number of noise scales go to infinity and the perturbation becomes a diffusion $dx = f(x, t)dt + g(t)dw$, whose reverse is

$$dx = [f(x, t) - g^2(t) \nabla_x \log p_t(x)]dt + g(t) d\bar{w}$$

so a time-dependent score model $s_\theta(x, t)$ is all that is needed to run the generative direction. NCSN is the **variance-exploding** SDE $dx = \sqrt{d[\sigma^2(t)]/dt} dw$; DDPM is the **variance-preserving** SDE $dx = -\frac{1}{2}\beta(t)x dt + \sqrt{\beta(t)} dw$.

And conditioning is one extra term. Because $\nabla_x \log p(x | y) = \nabla_x \log p(x) + \nabla_x \log p(y | x)$, an unconditional score model plus any differentiable oracle gives the conditional score — which is classifier guidance, and the reason a single trained prior over designs can be steered toward any property you can score.

Backup 5 — MINs and CbAS, in full

MINS — TRAIN, THEN QUERY

Train. $L_p(D) = \mathbb{E}_{y \sim p(y)} [\text{Div}(p_D(x | y), p_{f_\theta^{-1}}(x | y))]$, with KL $\Rightarrow$ maximum likelihood and JS $\Rightarrow$ adversarial training, reweighted by $w_i = p(y)/p_D(y)$, $p(y) \propto \frac{N_y}{N_y + K} e^{y - y^*}$.

The weight comes from a bias–variance bound whose three terms are *variance from thin data* $\mathbb{E}_y[1/N_y]$, *distribution shift* $d_2(p||p_D)/|D|$, and *bias from the target* $D_{\text{TV}}(p^*, p)^2$.

Query. $x = f_\theta^{-1}(z, y_{\text{target}})$, $z \sim \mathcal{N}(0, I)$, with y_{target} chosen by Approx-Infer. Ablations: on CIFAR-10 (uniform) MIN scores 77.12 against 76.31 without Approx-Infer and 74.87 without reweighting — **both components are load-bearing**.

The subtlety both share. Querying *above* the observed range is extrapolation — the same off-distribution hazard as Lecture 5, moved from input space into output space. Practical systems temper y_{target} and re-rank the samples with a (conservative) forward model, which marries the two halves of Part III into one pipeline: **generate to stay valid, score to select**.

CBAS — THE ADAPTIVE LADDER

```

 $\theta^0 \leftarrow$  fit the prior  $p(x|\theta^0)$  to D
 $\gamma^0 \leftarrow$  Q-th percentile of the oracle's
    predictions on the prior's samples
 $q \leftarrow p(\cdot|\theta^0)$ 
repeat:
     $x_1 \dots x_M \sim q$   $\leftarrow$  propose
     $\gamma \leftarrow \min(\text{target}, \text{Q-th pct of}$ 
         $\hat{f}(x_1 \dots x_M))$   $\leftarrow$  relax
     $w_i \leftarrow p(x_i|\theta^0)/q(x_i) \cdot P(y \geq \gamma | x_i)$ 
     $q \leftarrow$  weighted MLE on  $\{x_i, w_i\}$   $\leftarrow$  refit

```

The importance weight uses the *previous* iterate as the proposal, which is what keeps the estimator's variance finite; and γ rises only as fast as the model can follow, which is condition (B).